

Week 4 Report: Software-in-the-Loop (SIL) Validation

1. System Architecture & MQTT Facilitation (Task 1)

The Phase 1 Software-in-the-Loop (SIL) simulation connects a Python control signal publisher, a SysML Model representing the System of Interest (SoI), and a Python receiver. Communication uses public broker test.mosquitto.org:1883. The SoI subscribes to 20-byte payloads (five Float32 joint angles) on [validatecontrolsignals/roboticarm](#), validates joint bounds, and transmits approved signals on [transmitcontrolsignals/roboticarm](#).

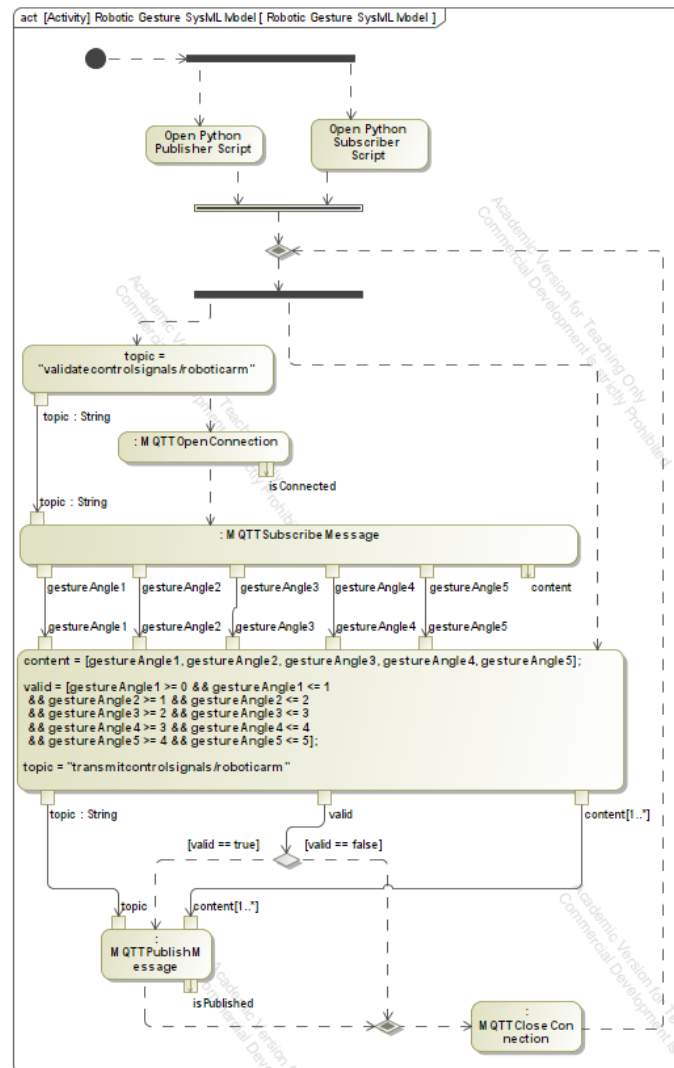


Figure 1: Task 1 Base SysML Activity Diagram Workflow

2. Signal Range Validation Logic & Decision Flow (Task 2)

An Opaque Action validates received control signals against physical joint bounds prior to forwarding. A Decision Gateway then routes execution based on boolean flag **valid**:

Joint Parameter	Validation Guard Logic Condition
Joint Angle 1	<code>gestureAngle1 >= 0 && gestureAngle1 <= 1</code>
Joint Angle 2	<code>gestureAngle2 >= 1 && gestureAngle2 <= 2</code>

Joint Parameter	Validation Guard Logic Condition
Joint Angle 3	<code>gestureAngle3 >= 2 && gestureAngle3 <= 3</code>
Joint Angle 4	<code>gestureAngle4 >= 3 && gestureAngle4 <= 4</code>
Joint Angle 5	<code>gestureAngle5 >= 4 && gestureAngle5 <= 5</code>

Decision Gateway Rules:

- `[valid == true]`: Executes `MQTTPublishMessage`, sending payload on `transmitcontrolsignals/roboticarm`.
- `[valid == false]`: Bypasses `MQTTPublishMessage`, executes `MQTTCloseConnection`, and loops to Merge Node for new signals.

3. Experimental Verification

SIL execution was verified by matching SysML console outputs with Python receiver logs across public MQTT topics.



Figure 3 & 4: SysML Output Array vs. Python Subscriber Received Array

Parameter	SysML Produced Value	Python Received Value	Verification Status
Joint Angle 1	0.9062	0.906	PASSED
Joint Angle 2	1.1878	1.188	PASSED
Joint Angle 3	2.0994	2.099	PASSED
Joint Angle 4	3.9964	3.996	PASSED
Joint Angle 5	4.0597	4.060	PASSED

4. Summary

Phase 1 SIL validation successfully confirms end-to-end signal communication and guard logic enforcement across all 5 joint parameters, meeting all requirements for Tasks 1 and 2.